

Tank 1990 тоглоомын Python/Pugame дээрх хэрэгжилт: Санамсаргүй газрын зураг үүсгэлт ба хиймэл оюун ухаант дайсны логик

1st Б. Баяр-Эрдэнэ

Хиймэл оюун ухааны инженер

Оюутны код: B251960001

Улаанбаатар хот, Монгол Улс

bayarb0329@gmail.com

2nd Э. Алтанбаяр

Хиймэл оюун ухааны инженер

Оюутны код: B251960029

Улаанбаатар хот, Монгол Улс

enhshaihanaltanbayr99@gmail.com

3rd Ц. Амгаланбаатар

Хиймэл оюун ухааны инженер

Оюутны код: B251960020

Улаанбаатар хот, Монгол Улс

tsogoo17amka@gmail.com

4th Г. Гантөгс

Хиймэл оюун ухааны инженер

Оюутны код: B251960022

Улаанбаатар хот, Монгол Улс

Ggganaa974@gmail.com

Abstract—Энэхүү өгүүлэлд “Battle City” буюу “Tank 1990” сонгодог аркад тоглоомын Python хэл болон Pugame сангийн тусламжтайгаар хийгдсэн хэрэгжилтийг судалж, тайлбарлав. Тухайлбал, тоглоомын архитектур, дайсны танкийг удирдах BotAI классын шийдвэр гаргах логик, дуу авиаг процедурын аргаар (procedural audio synthesis) бодит цагийн горимд үүсгэх арга барил, мөн тоглоомын байгуулалт (map)-ыг тоглолт бүрт санамсаргүйгээр шинээр үүсгэдэг алгоритмыг дэлгэрэнгүй авч үзнэ. Мөн бид энэ ажлын явцад илэрсэн хоёр гол асуудлыг — (1) газрын зураг тогтмол давтагддаг байсныг, мөн (2) бот танкны хөдөлгөөний чиглэл ба хараа чиглэл зөрдөг “чиг баримжаа алдагдал”-ыг — хэрхэн шийдвэрлэсэн тухай тодорхойлно.

Index Terms—Pugame, тоглоом хөгжүүлэлт, хиймэл оюун ухаан, санамсаргүй газрын зураг үүсгэлт, Battle City, Tank 1990

I. Оршил

Видео тоглоом хөгжүүлэлт нь програм хангамжийн инженерчлэлийн олон салбарыг (график дүрслэл, төлөв удирдлага, хиймэл оюун ухаан, дуу авиа боловсруулалт) нэгэн дор шаарддаг практик хэрэглээний чиглэл юм. Энэхүү ажилд бид 1990 онд Namco компанийн гаргасан “Battle City” тоглоомын сүнс дээр үндэслэн, Python хэл болон Pugame (community edition) сан ашиглан бичигдсэн клон тоглоом (tank1990.v1.3.0.py) -ийн дотоод

бүтэц, логикийг судалж, түүнд хийсэн сайжруулалтуудыг тодорхойлно.

Уг тоглоом нь 26×24 нүдтэй (tile-based) талбай дээр тоглогддог бөгөөд тоглогч (1 буюу 2 хүн хамтдаа) баазаа (fort) дайсны танкуудаас хамгаалах зорилготой. Дайсан танкууд BotAI классаар удирддаг бөгөөд энэ нь тоглогчийг хайж олох, хөөх (chase), тэнэх (patrol) болон буудах зэрэг зан үйлийг хэрэгжүүлдэг.

II. Тоглоомын ерөнхий архитектур

Тоглоомын код нь дараах үндсэн бүрэлдэхүүн хэсгүүдээс бүрдэнэ:

- **Тохиргоо ба тогтмолууд** — TILE хэмжээ, ROWS/COLS тоо, color palette, DIFFICULTY (хялбар/энгийн/хэцүү) тохиргооны толь бичиг, бот тус бүрийн бодох давтамж, буудах магадлал, хурдны илтгэгч зэрэг.
- **Дуу авиа модуль** — _make_wav, make_shoot_sound, make_explode_sound, make_hit_sound функцууд нь дуу авиаг урьдчилан бэлтгэсэн файлаас биш, харин синусоид болон шуугиан (noise) дохиог математикийн аргаар бодит цагт нэгтгэн (synthesize) pygame.mixer.Sound объект болгон үүсгэдэг.
- **Газрын зураг үүсгэлт** — build_map() функц, доор дэлгэрэнгүй авч үзнэ.
- **Дүрслэлийн функцууд** — draw_brick, draw_steel, draw_fort, draw_tank ГЭХ МЭТ,

тоглоомын элемент бүрийг пиксель түвшинд Pygame-ийн draw функцүүдээр зурдаг.

- **Bullet, Tank классууд** — сумны болон танкны байрлал, хурд, мөргөлдөөний rect, зурах логикийг агуулна.
- **BotAI класс** — дайсны танкийг удирдах шийдвэр гаргах логик (доор дэлгэрэнгүй).
- **Game класс** — үндсэн тоглоомын давталт (game loop): гар оролт, мөргөлдөөний илрүүлэлт, төлөв шинэчлэлт, зурах дараалал.
- **main()** — Pygame-ийг эхлүүлж, дэлгэцийн горим, цонхны хэмжээ, цагийн давтамжийг (FPS) тохируулан үндсэн давталтыг ажиллуулна.

III. BotAI классын шийдвэр гаргах логик

Дайсны танк бүр өөрийн BotAI объекттой холбогддог бөгөөд энэ нь tank, diff (хүндрэлийн тохиргоо), think_timer, move_dir, stuck_timer, patrol_timer гэсэн төлөвүүдийг агуулна. Кадр (frame) бүрт дуудагдах update() функц дараах алхмуудыг гүйцэтгэдэг:

- 1) **Гацсан эсэхийг илрүүлэх** — танкны байрлал өмнөх кадртай бараг ижил хэвээр байвал stuck_timer нэмэгдэж, 30 кадрын дараа санамсаргүй шинэ чиглэл сонгоно.
- 2) **Шийдвэр гаргах давтамж** — think_timer 0 болмогц хамгийн ойрхон амьд тоглогчийг Манхэттений зайгаар ($|dx| + |dy|$) тодорхойлж, хэрэв тэр chase_dist-ээс дотогш байвал түүн рүү чиглэн хөдлөх (chase), эсрэг тохиолдолд санамсаргүйгээр тэнэх (patrol) шийдвэр гаргана.
- 3) **Хөдөлгөөн** — сонгосон чиглэлээр t.move() функцээр танкийг хөдөлгөнө; энэ функц нь мөн танкны харах чиглэлийг (t.dir) хөдөлгөөний чиглэлтэй адилтгана.
- 4) **Буудах логик** — хамгийн ойрхон тоглогчтой эгнэсэн (aligned) эсэх, мөн шулуун харагдац (line-of-sight, хананд бөглөрөөгүй эсэх)-г шалгаж, тохирвол буудна; эс бол бага магадлалаар (bot_shoot_chance) санамсаргүй буудна.

A. Илэрсэн асуудал: чиг баримжаа зөрчил

Анхны хувилбарт буудах логик нь t.dir-ийг зорилтот тоглогч руу чиглүүлэн тусад нь албадан өөрчилдөг байсан бол хөдөлгөөний чиглэл (move_dir) өөр байх боломжтой байв. Үүний үр дүнд танк, жишээлбэл, баруун тийш хөдөлж байхдаа хошуугаа доош харуулах мэт харааны алдаа гарч байв. Бид энэ асуудлыг t.dir-ийг буудах мөчид дахин бичихгүй,

харин аль хэдийн хөдөлгөөнөөр тогтоогдсон одоогийн харах чиглэлийг ашиглан зорилтотой эгнэсэн эсэхийг шалгах байдлаар засварласан. Ингэснээр танкны хараа чиглэл хөдөлгөөнөөс хэзээ ч зөрөхгүй болсон.

IV. Санамсаргүй газрын зураг үүсгэлт

Анхны хувилбарт build_map() функц нь BASE_MAP нэртэй тогтмол текстэн загвараас (26 мөр, тэмдэгтээр “#”=тоосго, “S”=ган, “F”=бааз) grid үүсгэдэг байсан тул тоглолт бүрт яг ижил байгууламжтай газрын зураг давтагддаг байв. Бид энэхүү функцийг тоглолт бүрт өөр байгууламж санамсаргүйгээр үүсгэдэг болгож дараах зарчмуудын дагуу дахин бичив:

- **Баазын бүтэц тогтмол хэвээр** — win/lose логик болон spawn цэгүүдийн зөв ажиллагааг хадгалахын тулд баазын (fort) байрлал, хэлбэрийг үргэлж ижил байлгана.
- **Аюулгүй коридорууд** — тоглогч (P1, P2) болон дайсны spawn цэгүүдийн ойролцоох баганууд, эхний болон сүүлийн мөрүүд үргэлж хоосон үлдэж, spawn хийхэд ханан дотор гацахаас сэргийлнэ.
- **Тэгш хэмт санамсаргүй байдал** — үлдсэн талбайг зүүн-баруун тэгш хэмтэйгээр ($tx \leftrightarrow COLS - 1 - tx$) тоосго (55%), ган (13%) эсвэл хоосон (32%) магадлалаар дүүргэж, тоглоомыг тэнцвэртэй, шударга байлгана.
- **Хөдөлгөөний коридорууд** — тодорхой давтамжтай мөрүүдийг (0, 4, 8, 12, 16, 20) бүрэн хоосон үлдээж, талбай бүхэлдээ хана болж бөглөрөхөөс сэргийлнэ.

Энэхүү өөрчлөлтийн үр дүнд random.seed() өөр байх тутам огт өөр байгууламжтай, гэвч тоглогдох боломжтой газрын зураг үүсдэгийг туршилтаар (Хэсэг VI) баталгаажуулав.

V. Хувилбарын түүх ба нэмэлт өөрчлөлтүүд

Төслийн хөгжүүлэлтийн явцад код нь хэд хэдэн хувилбараар дамжин боловсронгуй болсон бөгөөд эцсийн хувилбар нь tank1990.v1.3.0.py нэртэй болов (Python 3.14, pygame-ce сан ашиглав, локал Python interpreter дээр, виртуал орчингүйгээр туршигдсан). Хувилбар тус бүрт хийгдсэн нэмэлт, өөрчлөлт, засварыг доор он дараалалтайгаар нэгтгэв.

A. Хувилбар 1.1.0

- **Засвар:** Удирдлагын (controller) алдааг засав.

В. Хувилбар 1.2.0

- **Нэмэлт:** Тоглогч цонхны (tab) хэмжээг өөрчлөх боломжтой болов.
- **Нэмэлт:** Бот танкны буудах хугацаанд хойшлуулалт (bot cooldown) нэмэв.
- **Нэмэлт:** Дэлгэцийн баруун доод буланд хөгжүүлэгчдийн нэрсийг харуулав.

С. Хувилбар 1.2.1

- **Илэрсэн алдаа:** Өмнөх хувилбарт бот танк баазыг (fort) санамсаргүйгээр буудахгүй байх, мөн бааз бүрэн устсан ч тоглоом дуусахгүй үлддэг логик болон механик алдаа илэрсэн.
- **Засвар:** Дээрх логик болон механикийн алдааг засаж, баазыг устгасны дараа тоглоом зөв дуусдаг болгов.

Д. Хувилбар 1.2.2

- **Нэмэлт:** Тоглогч Game Over дэлгэц дээрээ өөрийн авсан оноогоо (score) харах боломжтой болов.
- **Нэмэлт:** Тоглоом дуусах (GAME END) дэлгэц дээр хамгийн өндөр оноог (best score) харуулах болов.
- **Нэмэлт:** Тоглогчийн танк одоо хамгаалалтын бамбай (shield) -тай төрдөг болов.

Е. Хувилбар 1.3.0 (эцсийн хувилбар)

- **Засвар:** Бот танкны хөдөлгөөний логикийг засварлав (Хэсэг III-A-д дэлгэрэнгүй тайлбарласан чиг баримжааны зөрчлийн засвар).
- **Нэмэлт:** Өмнө нь тоглоом зөвхөн нэг тогтмол газрын зурагтай байсан бол одоо тоглоом эхлэх бүрд газрын зураг санамсаргүйгээр шинээр үүсгэгддэг болов (Хэсэг VI).

Дээрх өөрчлөлтүүдийн ерөнхий агуулгыг Хүснэгт I-д хураангуйлав.

TABLE I
Хувилбарын түүхийн хураангуй

Хувилбар	Гол өөрчлөлт
1.1.0	Хянагчийн (controller) алдааг засав
1.2.0	Цонхны хэмжээ өөрчлөх, bot cooldown, хөгжүүлэгчийн нэрс
1.2.1	Баазын устгал/тоглоом дуусах логикийн алдааг засав
1.2.2	Оноо, хамгийн өндөр оноо харуулах, бамбайтай төрөх
1.3.0	Бот хөдөлгөөний засвар, санамсаргүй газрын зураг

VI. Туршилт ба үр дүн

Бид random.seed()-ийн өөр өөр утгаар build_map() функцийг олон удаа дуудаж, дараах шинж чанаруудыг шалгав:

- Grid-ийн хэмжээ үргэлж 24×26 байгаа эсэх.
- Баазын байрлал (мөр 22-23, багана 10-14) бүх дуудалтад ижил хэвээр байгаа эсэх.
- Хоёр тоглогчийн spawn цэг (багана 9 ба 15, мөр 22) үргэлж хоосон байгаа эсэх.
- Дайсны spawn мөр (мөр 0) бүхэлдээ хоосон байгаа эсэх.
- Өөр seed утгаар үүсгэсэн хоёр grid бие биенээсээ ялгаатай байгаа эсэх.

Бүх шалгалт амжилттай давагдсан бөгөөд энэ нь функц зорилтот шаардлагыг (spawn/бааз хамгаалагдсан, боловч давхардаагүй санамсаргүй байгууламж) хангаж байгааг харуулна.

А. Тоглогч ба bot-ийн тоглолтын судалгаа

Тайлбар: Судалгааг цаашид Хэсэг III-A-д дурдсан DIFFICULTY гурван түвшин (easy, normal, hard) тус бүр дээр тоглогчдыг хэд хэдэн удаа тоглуулж, win/loss болон үхсэн bot-ийн түвшинг бүртгэх байдлаар явуулсан болно.

Судалгааны хүрээнд 10 тоглогч оролцож, тоглогч бүр гурван хүндрэлийн түвшин дээр тус бүр 3 удаа, нийт 90 тоглолт хийсэн ба үр дүнг Хүснэгт II-д харуулав.

TABLE II
Судалгаа: хүндрэлийн түвшин тус бүрээрх хожил/хожигдлын харьцаа (n=10 тоглогч, тус бүр 3 тоглолт)

Түвшин	Хожсон	Хожигдсон	Хожих %
Easy	25	5	84%
Normal	16	14	54%
Hard	5	25	18%

Мөн тоглогч тус бүрээс аль хүндрэлийн түвшний bot дээр хамгийн олон удаа (магадгүй эсвэл давтан) үхсэн талаар асуусан судалгааны үр дүнг Хүснэгт III-д харуулав.

TABLE III
Судалгаа: тоглогчдын хамгийн олон удаа үхсэн хүндрэлийн түвшин (n=10 тоглогч)

Түвшин	Үхсэн тоо	Эзлэх хувь
Easy	60	21,4%
Normal	91	32,5%
Hard	129	46%

Дээрх өгөгдлөөс харахад хүндрэлийн түвшин өсөх тутам тоглогчийн хожих магадлал буурч, hard түвшин дээр хамгийн олон удаа үхсэн харагдаж байна. Энэ нь DIFFICULTY

тохиргоон дахь `bot_shoot_chance`, `chase_dist` болон `bot_speed_mul` зэрэг параметрууд түвшин ахих тутам нэмэгддэгтэй (Хэсэг III-A) чанарын хувьд нийцтэй үр дүн юм.

VII. Дүгнэлт

Энэхүү өгүүлэлд бид Python/Pygame дээр хэрэгжүүлсэн Tank 1990 клон тоглоомын архитектурыг тайлбарлаж, түүний дотор дайсны танкийг удирдах BotAI классын шийдвэр гаргах логикийг дэлгэрэнгүй авч үзэв. Мөн бид тоглоомд илэрсэн дутагдлуудыг — газрын зураг үргэлж давтагддаг байсныг санамсаргүй үүсгэлтээр, мөн бот танкны хараа чиглэл хөдөлгөөнөөс зөрдөг асуудлыг чиг баримжааг зөвхөн нэг эх сурвалжаас (хөдөлгөөнөөс) авдаг болгосноор, удирдлагын (controller) алдааг, тоглогч цонхны (tab) хэмжээг өөрчлөх боломжтой болгох, `bot cooldown` нэмэх, бот танк баазыг (fort) санамсаргүйгээр буудахгүй байх, мөн бааз бүрэн устсан ч тоглоом дуусахгүй үлддэг логик болон механик алдааг — амжилттай засварлав. Ирээдүйд олон түвшний (stage) прогресс, мөн бот AI-д машин сургалт суурилсан шийдвэр гаргалт нэмэх боломжтой.

Талархал

Энэхүү ажлыг гүйцэтгэхэд зөвлөгөө, дэмжлэг үзүүлсэн бүх хүмүүст талархал илэрхийлье. Уг төслийг Б. Баяр-Эрдэнэ, Э. Алтанбаяр, Ц. Амгаланбаатар, Г. Гантөгс нар хамтран хөгжүүлэв.

References

- [1] Namco, “Battle City,” Famicom тоглоом, 1985.
- [2] Pygame Community Edition баг, “Pygame-ce Documentation.” [Онлайн]. Хандах линк: <https://pyga.me/docs/>
- [3] Python Software Foundation, “The Python Standard Library — random module.” [Онлайн]. Хандах линк: <https://docs.python.org/3/library/random.html>